Claude Manager: что нового

Руководство пользователя по изменениям 27–28 октября 2026 года — просто и по-дружески

Привет! За вечер 27 октября в менеджер приехал большой блок **«Экономия и контроль»**. Его цель простая: агенты тратят меньше впустую, а ты видишь главное, не открывая каждую сессию. 28 октября всё это вывели в интерфейс: теперь каждая функция включается галочкой в настройках, а результаты видны на вкладках.

Главное, что нужно знать

- **Почти всё выключено по умолчанию.** Ничего не поменяется, пока ты сам не включишь. Исключение — тариф: по умолчанию «Подписка».
- **Включаются без перезапуска:** отметил галочку → нажал **«Сохранить»** → работает.
- **В твой репозиторий ничего не пишется без твоей галочки** в интерфейсе (хуки, файлы выученного, задачи из предложений).
- **Красные рамки и номера** 1 на скриншотах показывают, куда смотреть и в каком порядке нажимать.

Скриншоты сняты с тестового экземпляра менеджера с демонстрационными данными (проекты «lumen» и «claude-manager»); цифры на них условные.

Шаг 0. Как получить новую версию

1. Дождись, пока сессии закончат текущие задачи (перезапуск менеджера останавливает все сессии).
2. Запусти `run-claude-manager.bat` — он сам соберёт новую версию через `wails build`.
3. Закрой старый менеджер через значок в трее → **Quit** (крестик окна только прячет его в трей) и запусти снова.

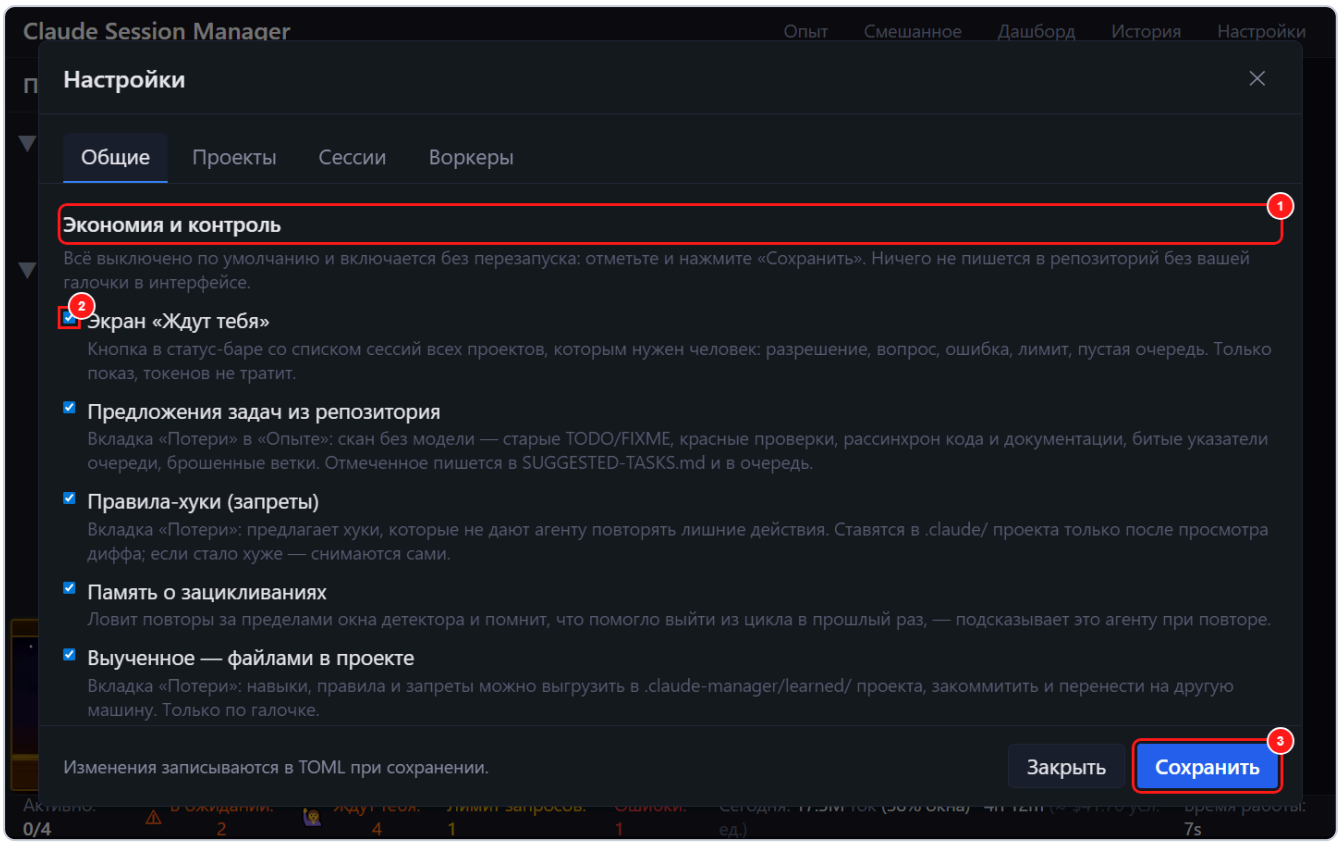
После этого в **Настройки** → **Общие** появится секция «Экономия и контроль», а в окне **«Опыт»** — вкладка «Качество».

Коротко: что где включается

Функция	Где включить	Где смотреть результат	Тратит токены?
Тариф и окно лимита	Настройки → Общие → «Тариф»	Статус-бар, карточка сессии, Дашборд, «Потери»	Нет
Отчёт о потерях	Включён всегда (нужен «Сбор опыта»)	Опыт → Потери	Нет
Экран «Ждут тебя»	Настройки → Общие → «Экономия и контроль»	Кнопка в статус-баре	Нет
Предложения задач из репозитория	то же	Опыт → Потери	Нет (без модели)
Правила-хуки (запреты)	то же	Опыт → Потери	Нет, даже экономит
Память о зацикливаниях	то же	Лог сессии (подсказки)	Нет
Выученное — файлами	то же	Опыт → Потери	Нет
Эталонные задачи	то же + проверки проекта	Опыт → Качество	Только перепрогон
Метки качества	то же	Опыт → Качество	Чуть-чуть (haiku, только на неудачах)
Бенчмарк конфигураций	то же	Опыт → Качество	**Да, много**
Строгие проверки	Настройки → Сессии → «Кругов доработки…»	Лог сессии, «Ждут тебя»	Да — круги доработки
Ночной прогон проверок	Настройки → Проекты → «Ночной прогон…»	Очередь, SUGGESTED-TASKS.md	Нет (сессии не запускает)

1. Где всё включается: Настройки

Настройки → Общие → «Экономия и контроль»

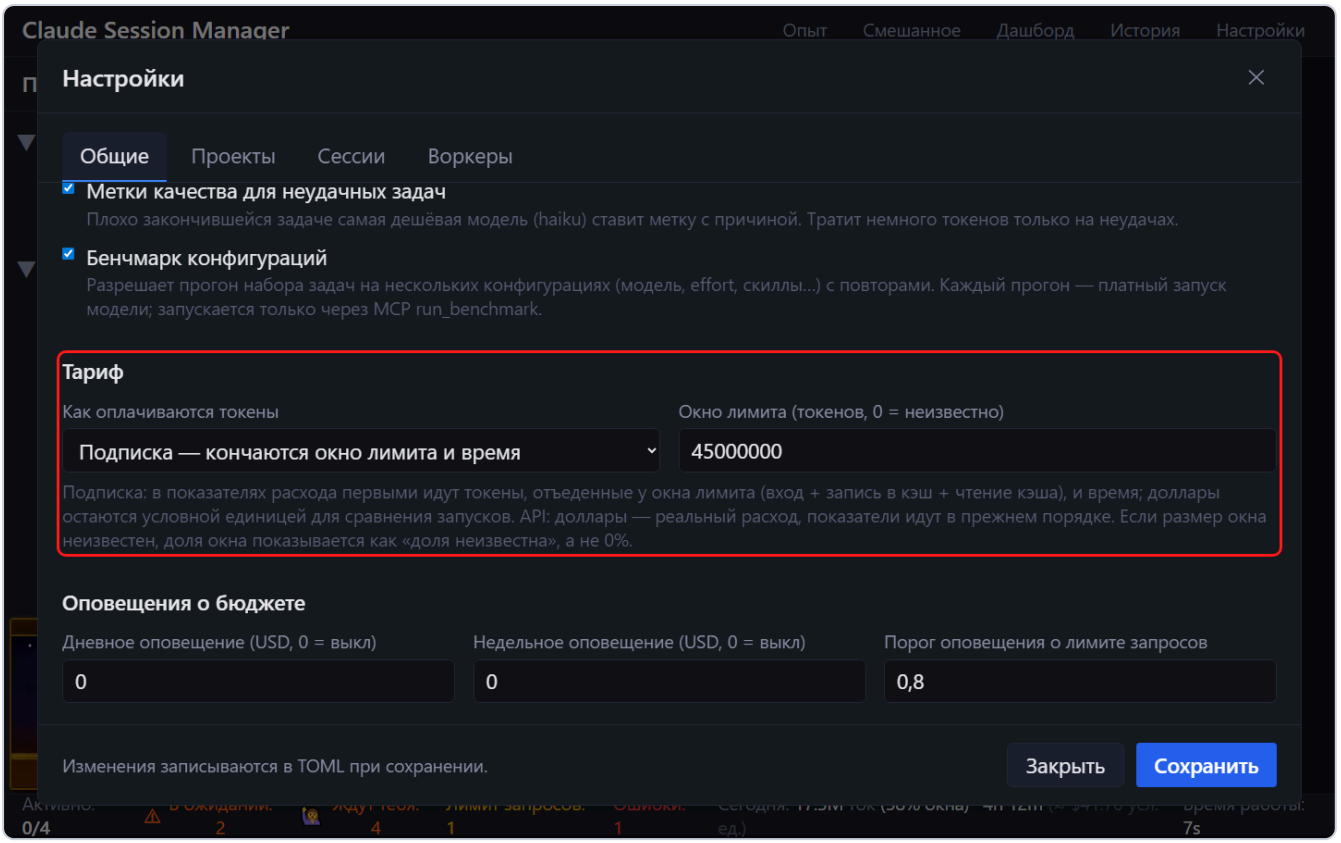


Секция «Экономия и контроль»: 1 заголовок секции, 2 галочки функций, 3 кнопка «Сохранить»

Открой **Настройки** (вверху справа) → вкладка **Общие** → прокрути до секции **«Экономия и контроль»**. Здесь восемь галочек. Под каждой — короткое пояснение, что она делает. Отметь нужные и нажми **«Сохранить»** 3.

> **Совет для старта**
> Включи «Экран „Ждут тебя“» и «Предложения задач из репозитория» — они безопасные, бесплатные и полезны каждый день. «Бенчмарк» не включай «на всякий случай» — он нужен только когда хочешь сравнить модели, и стоит денег.

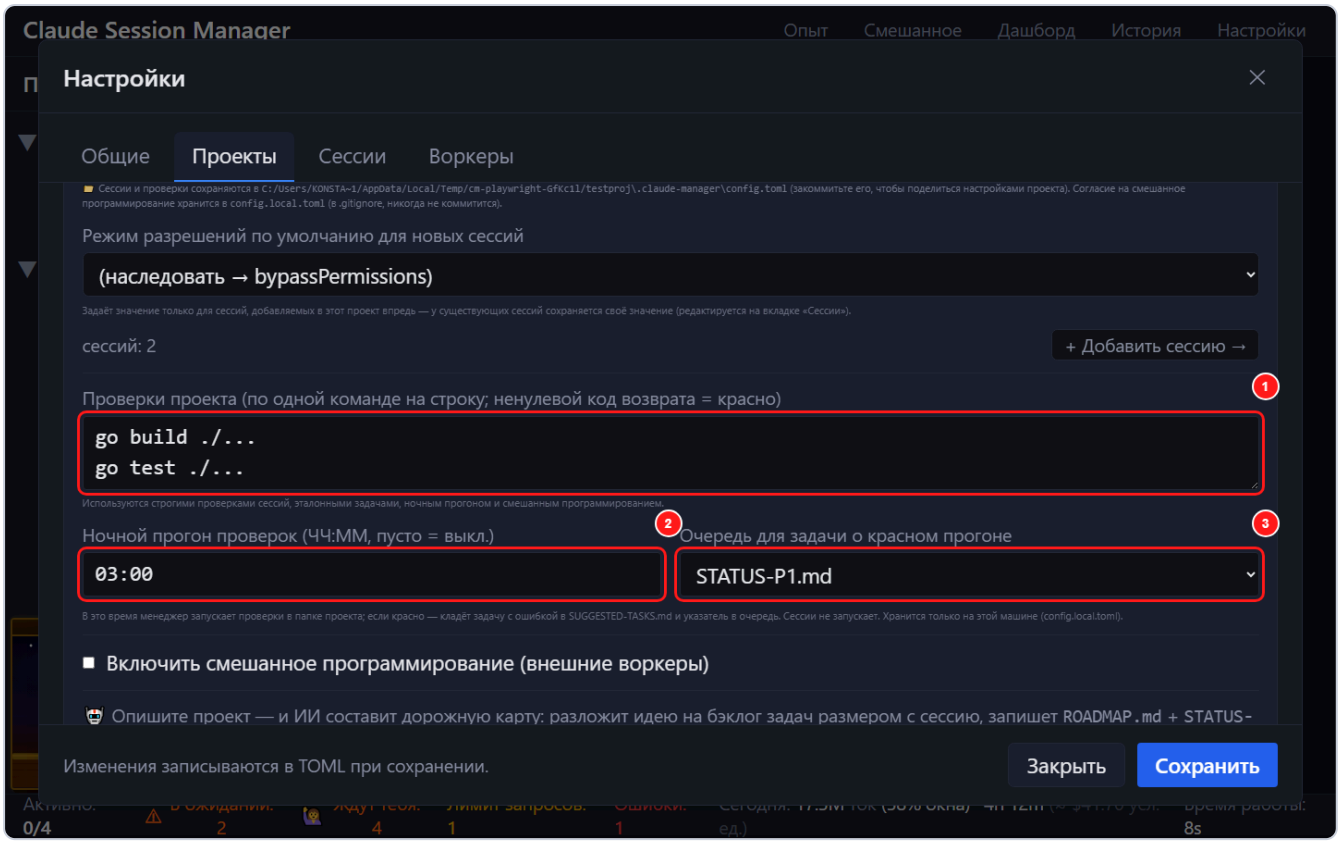
Настройки → Общие → «Тариф»



Секция «Тариф»: как оплачиваются токены и размер окна лимита

- **«Подписка»** (по умолчанию) — главное в расходе не доллары, а то, сколько ты съел из окна лимита, и время. Поэтому везде первыми показываются токены из окна лимита (вход + запись в кэш + чтение кэша) и время, а доллары остаются мелким шрифтом как «усл. ед.» — для сравнения запусков между собой.
- **«API»** — доллары это реальные деньги; всё выглядит как раньше.
- **«Окно лимита (токенов)»** — впиши размер своего окна, и рядом с расходом появится процент: «38% окна». Если оставить 0, будет честно написано «доля неизвестна», а не 0%.

Настройки → Проекты → проверки и ночной прогон

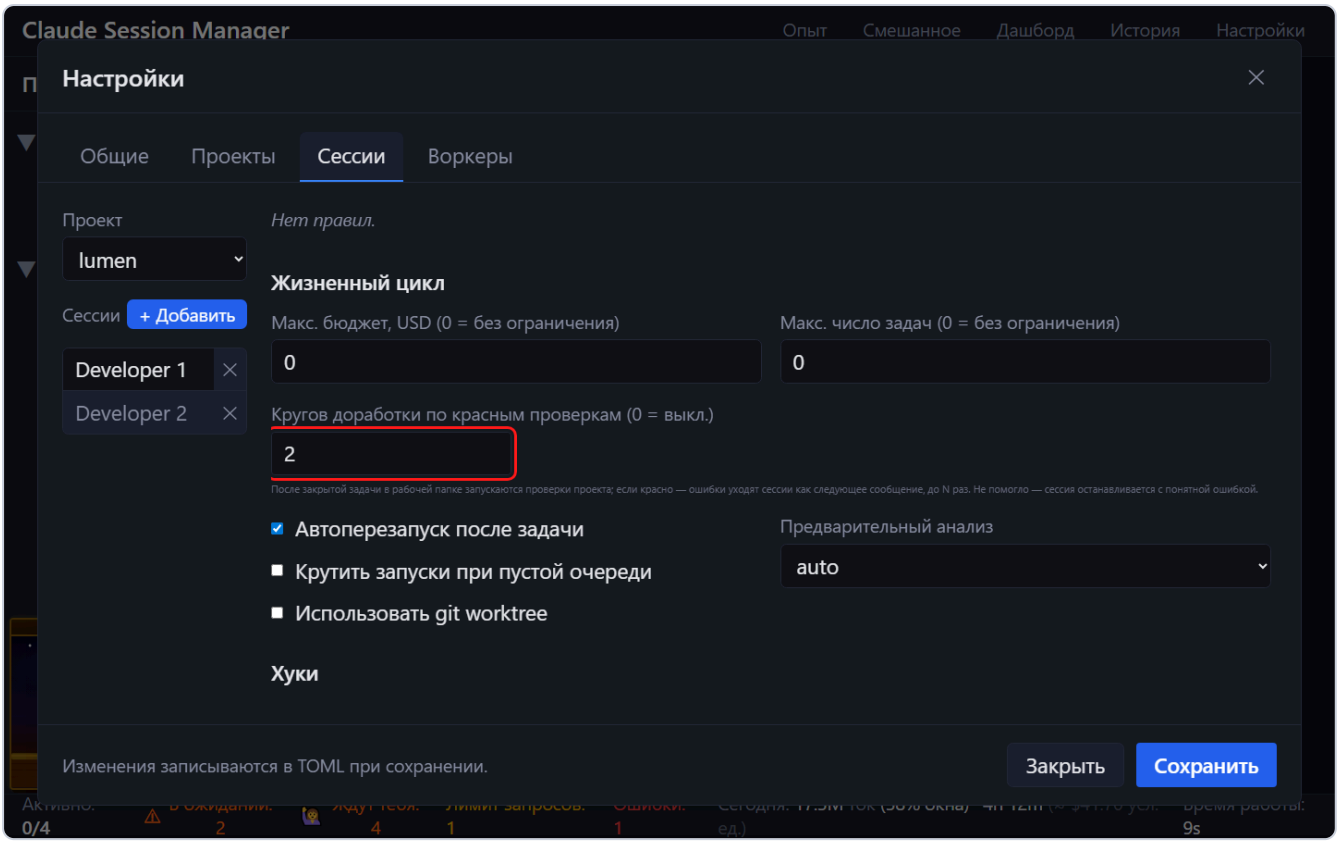


1 проверки проекта, 2 время ночного прогона, 3 очередь для задачи об ошибке

«Проверки проекта» 1 — команды, которыми проверяется, что проект в порядке, по одной на строку (например, `go build ./...` и `go test ./...`). Если команда вернула ненулевой код — проверки «красные». Раньше это поле было спрятано в блоке смешанного программирования; теперь оно отдельно, потому что на нём держатся строгие проверки, эталонные задачи и ночной прогон.

«Ночной прогон проверок» 2 и **«Очередь»** 3 — подробнее в разделе 9.

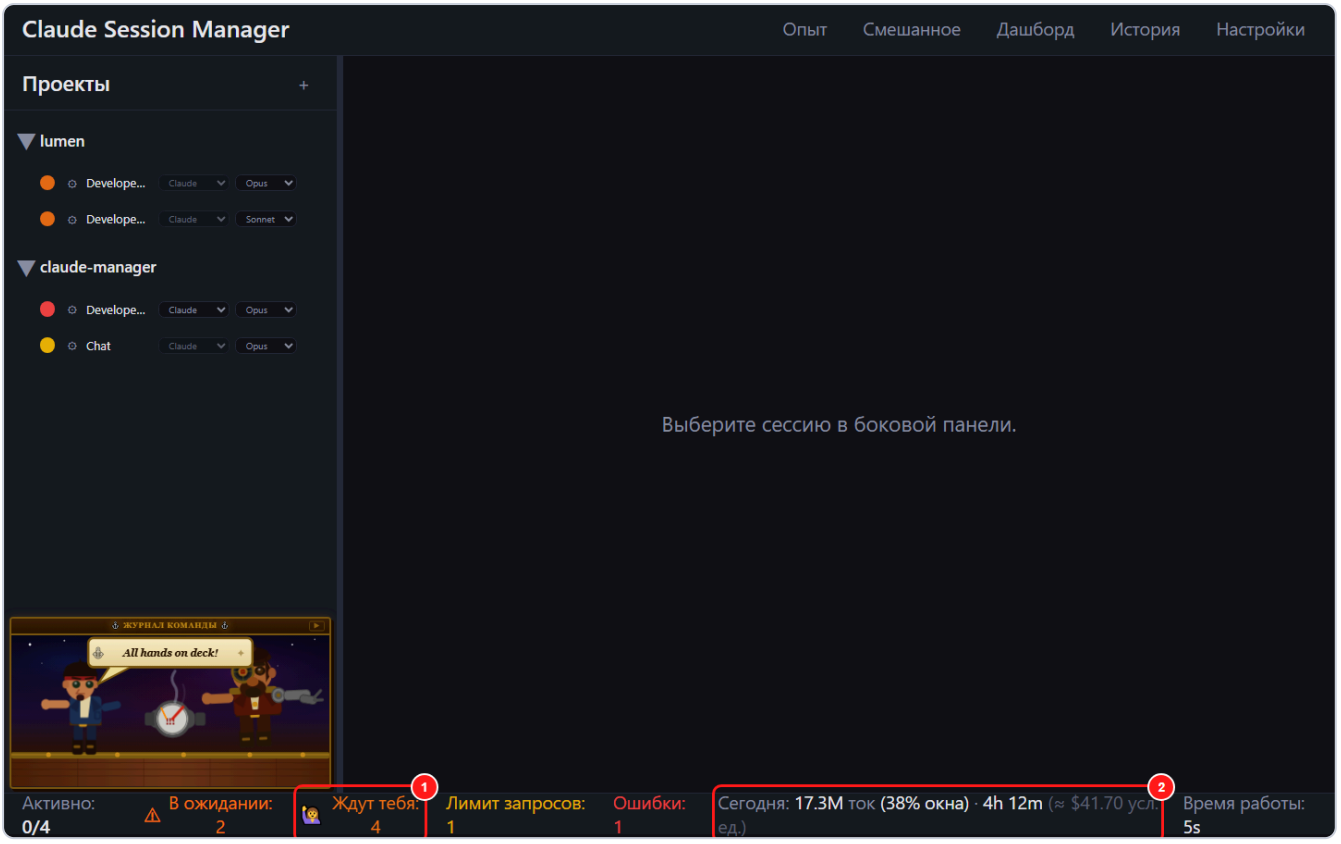
Настройки → Сессии → строгие проверки



Поле «Кругов доработки по красным проверкам» у выбранной сессии

Подробнее — в разделе 8. Коротко: 0 = выключено; 2 = после задачи прогнать проверки и, если красно, дать сессии до двух попыток исправить.

2. Статус-бар: расход по окну лимита и «Ждут тебя»

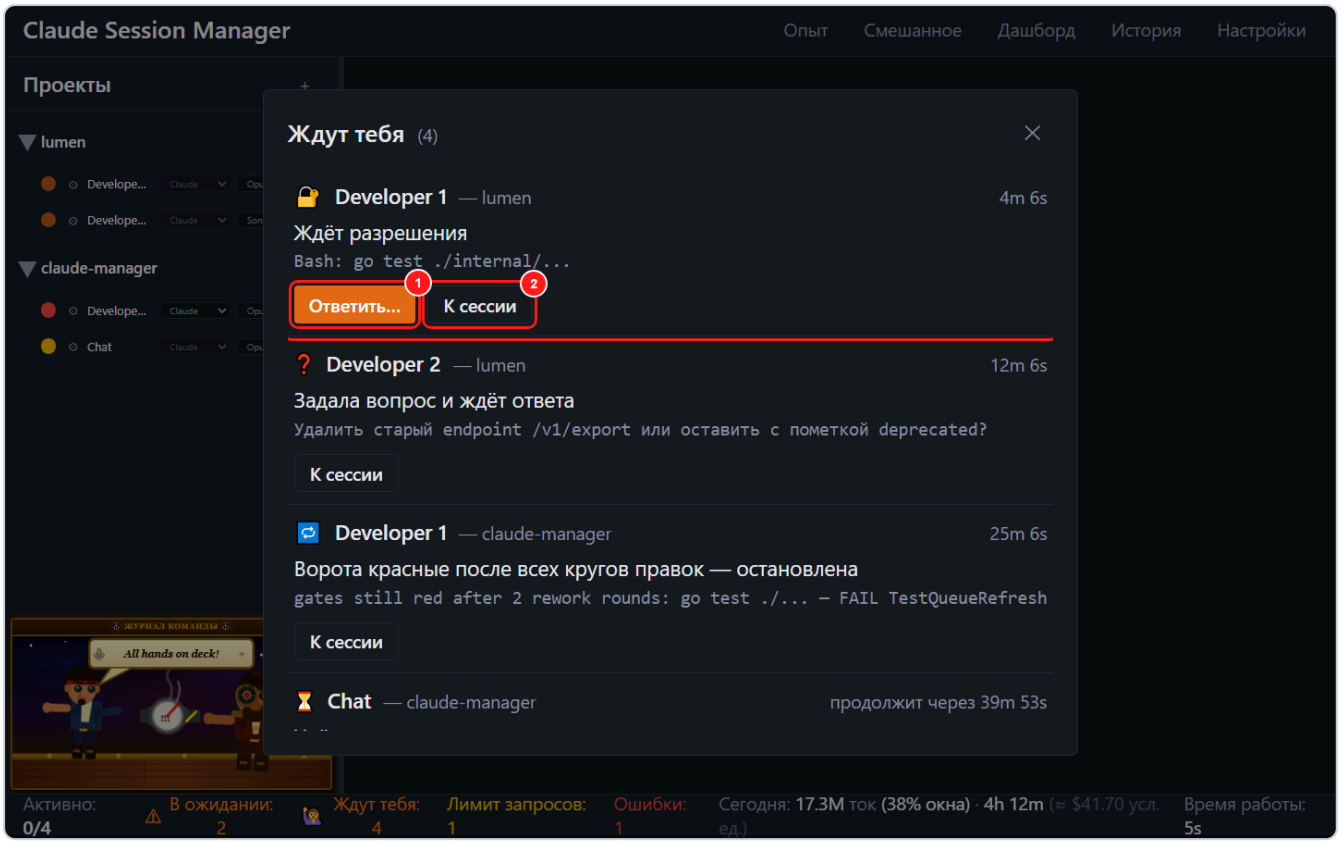


1 кнопка «Ждут тебя: 4», 2 «Сегодня»: токены, доля окна, время, доллары как условные единицы

Внизу окна теперь две новые штуки:

- 2 **«Сегодня: 17.3M ток (38% окна) · 4h 12m (≈ $41.70 усл. ед.)»** — сколько токенов из окна лимита ушло за день, какая это доля окна и сколько времени работали сессии. Такая же логика — на карточке сессии и в Дашборде.
- 1 **«Ждут тебя: N»** — появляется, только если включён «Экран „Ждут тебя“». Число — сколько сессий во всех проектах сейчас ждут именно тебя. Если там есть кто-то, кнопка оранжевая.

3. Экран «Ждут тебя»



Один список по всем проектам: 1 ответить на разрешение прямо отсюда, 2 перейти к сессии

Нажми кнопку в статус-баре — откроется один список всех, кому нужен человек, сразу по всем проектам. Самое срочное — сверху. Для каждой строки видно, **кто** (сессия и проект), **почему** и **сколько уже ждёт**.

Причины бывают такие:

Причина	Что делать
Ждёт разрешения	«Ответить...» 1 — откроется обычное окно разрешений
Задала вопрос и ждёт ответа	«К сессии» 2 и ответить в чате
Ворота (проверки) красные после всех кругов правок — остановлена	Посмотреть ошибку, поправить или дать указание
Остановилась с ошибкой	Посмотреть лог, перезапустить
Упёрлась в лимит	Ничего: видно, через сколько продолжит сама
Остановлена по запросу после задачи / очередь опустела	Дать новые задачи или запустить снова
Задача смешанного программирования требует человека	«Открыть смешанный запуск»

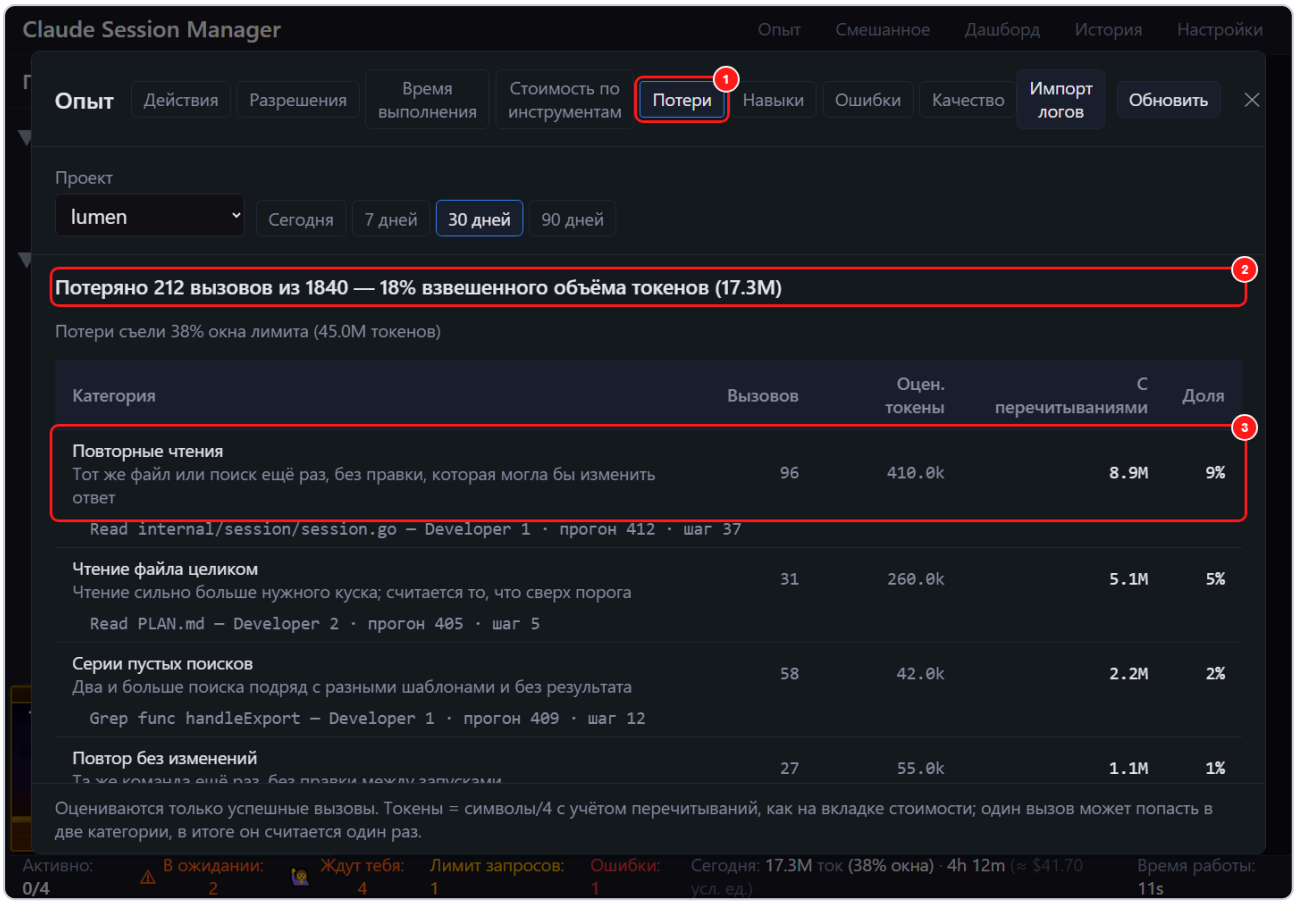
> **Как включить**
> Настройки → Общие → «Экономия и контроль» → **Экран «Ждут тебя»** → «Сохранить». Кнопка появится сразу. Это только экран: он ничего не запускает и токенов не тратит.

4. Окно «Опыт» → вкладка «Потери»

Открой **«Опыт»** (вверху справа), выбери **проект** в фильтре и вкладку **«Потери»**. Здесь собраны четыре блока: отчёт о потерях (всегда) и три блока, которые появляются, когда включены их галочки.

> **Обрати внимание**
> Без выбранного проекта блоки пустые — всё считается по проекту. А блоки «Правила-хуки», «Предложения» и «Выученное» просто не видны, пока их галочки выключены.

4.1. Отчёт о потерях



1 вкладка «Потери», 2 итог: сколько вызовов и токенов ушло впустую, 3 категории с примерами

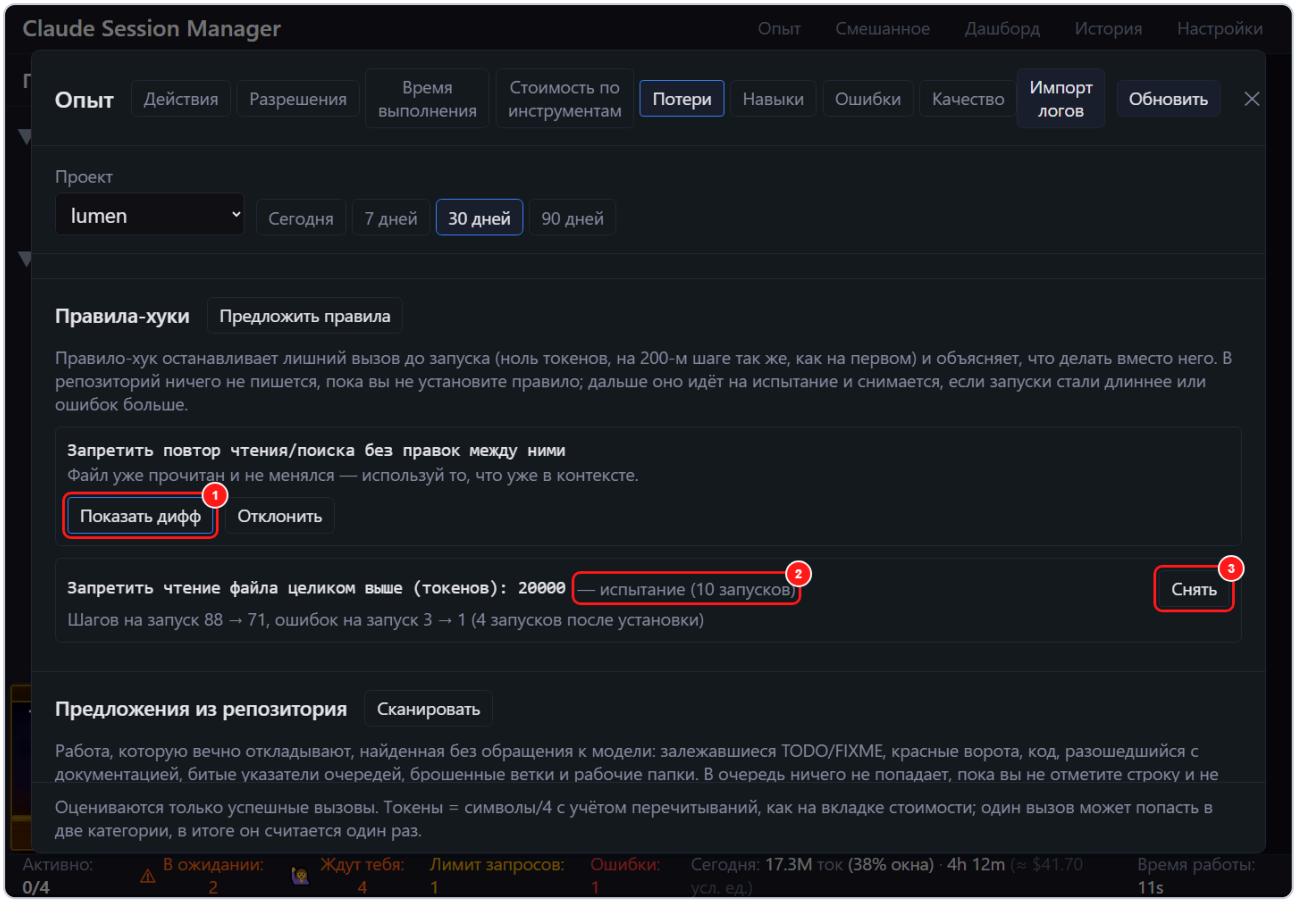
Это работа, которая **выглядела полезной и прошла успешно, но ничего не дала**. Считаются четыре вида:

- **Повторные чтения** — тот же файл или поиск ещё раз, хотя между ними ничего не правили.
- **Чтение файла целиком** — прочитано сильно больше, чем было нужно.
- **Серии пустых поисков** — два и больше поиска подряд без результата.
- **Повтор без изменений** — тот же `go test` ещё раз, хотя код не менялся.

Токены считаются «с перечитываниями»: всё, что попало в контекст, модель перечитывает на каждом следующем шаге, поэтому лишний большой файл в начале сессии стоит дорого. Строка 2 говорит общий итог, а при известном окне лимита — ещё и «Потери съели 38% окна лимита».

> **Как включить**
> Ничего делать не нужно — отчёт работает всегда, если включён «Сбор опыта» (он включён по умолчанию). Пороги можно подкрутить в конфиге: `waste_empty_search_chars`, `waste_oversized_read_tokens` (0 = встроенные значения).

4.2. Правила-хуки: запреты вместо уговоров



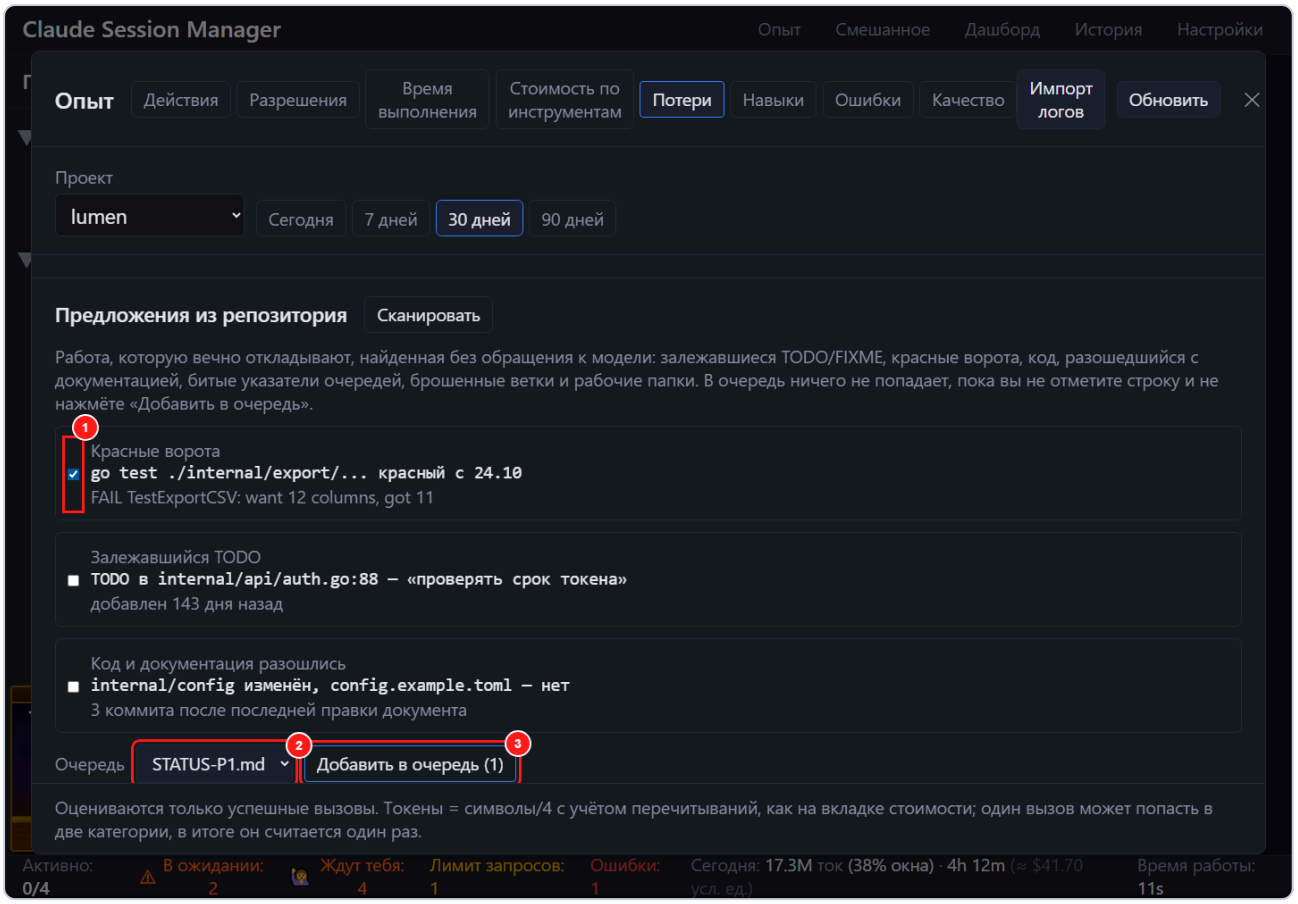
1 сначала посмотри дифф, 2 правило на испытании и его эффект, 3 снять правило вручную

Раньше выученное правило дописывалось в промпт — это «уговор»: он стоит токенов на каждом шаге и забывается к концу длинной сессии. **Хук** — это «запрет»: маленький скрипт, который останавливает лишний вызов **до** запуска и подсказывает агенту, что делать вместо этого. Стоит ноль токенов и на 200-м шаге работает так же, как на первом.

1. Нажми **«Предложить правила»** — менеджер подберёт их по истории проекта (запретить повтор чтения без правок, не гадать после двух пустых поисков, не читать огромные файлы целиком, не повторять команду, которая постоянно падает).
2. Нажми **«Показать дифф»** 1 — увидишь, какие файлы появятся в `.claude/` проекта.
3. Если согласен — **«Установить в репозиторий»**. Не нравится — «Отклонить».
4. Правило идёт на **испытание** 2 (10 запусков). Менеджер сравнивает шаги и ошибки на запуск «до» и «после». Если стало хуже — правило **снимается само**. Снять вручную — кнопка «Снять» 3.

> **На что влияет**
> Хук пишется в `.claude/` твоего проекта (это попадёт в git). Ставится только после твоего «Установить».

4.3. Предложения задач из репозитория



1 отметь нужное, 2 выбери очередь, 3 «Добавить в очередь»

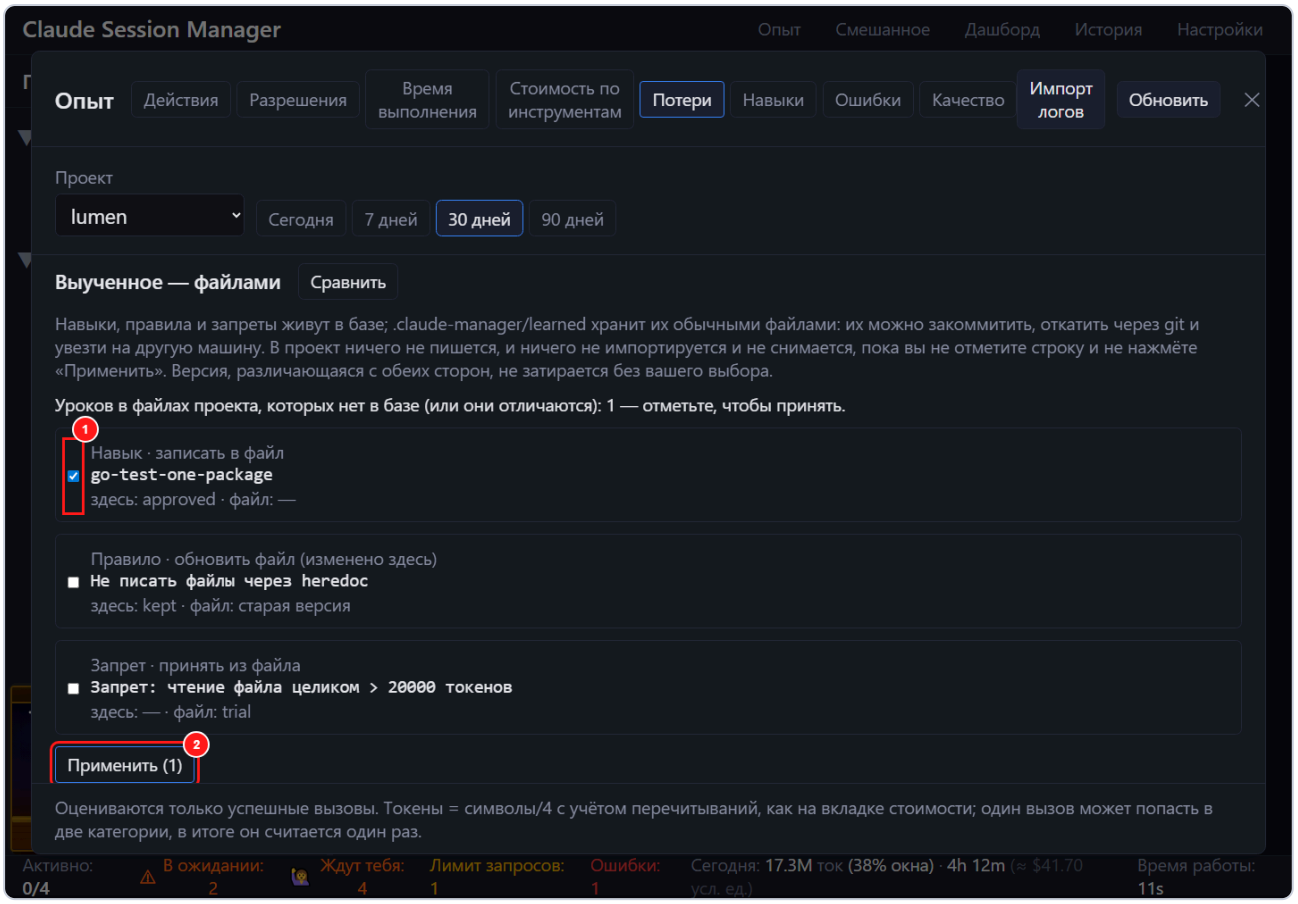
Нажми **«Сканировать»**. Менеджер **без модели** (бесплатно) найдёт работу, которую вечно откладывают:

- залежавшиеся TODO/FIXME (по умолчанию старше 90 дней — по git blame);
- красные проверки, на которые с тех пор никто не делал коммит;
- код, разошедшийся с документацией;
- битые указатели в очереди;
- брошенные ветки и рабочие папки (по умолчанию старше 7 дней).

Отметь нужные строки 1, выбери очередь 2 и нажми **«Добавить в очередь»** 3. Каждая отмеченная строка станет задачей `## SG-NN:` в файле `SUGGESTED-TASKS.md`, а её указатель допишется в конец выбранной очереди — разработчик возьмёт её в свой черёд. Уже добавленное при следующем скане повторно не предлагается.

> **Обрати внимание**
> Если у проекта нет ни одной сессии с очередью (источником задач), добавлять некуда — блок так и скажет. Порог «залежалости» меняется в конфиге: `suggest_stale_days`.

4.4. Выученное — файлами в проекте



1 отметь, что синхронизировать, 2 «Применить»

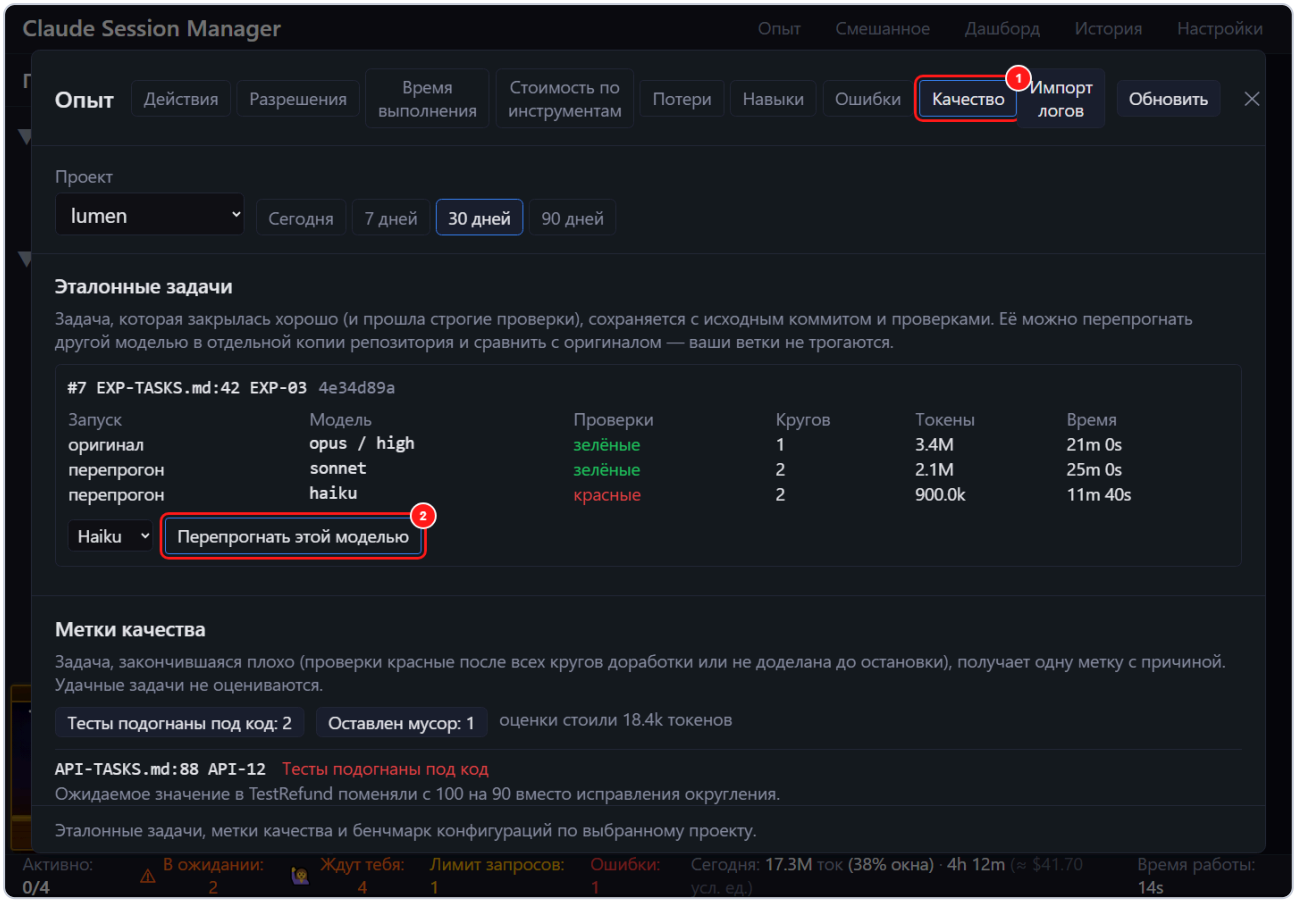
Навыки, правила и запреты, которые менеджер выучил, живут в его базе на этой машине. Этот блок позволяет хранить их **обычными файлами** в `.claude-manager/learned/` проекта: закоммитить, откатить через git, увезти на другой компьютер.

1. Нажми **«Сравнить»** — увидишь, что отличается между базой и файлами: «записать в файл», «обновить файл», «принять из файла», «файл удалён — снять здесь» и т.п.
2. Отметь нужные строки 1 и нажми **«Применить»** 2.
3. Если версия поменялась **с обеих сторон** — это «конфликт». Его никогда не перезапишет молча: выбери «Взять файл» или «Взять эту машину».

> **Приятная мелочь**
> Если откатить коммит с файлом урока — урок перестанет действовать. А в файлах записан измеренный эффект, и они детерминированы: в коммите видны только настоящие изменения.

5. Окно «Опыт» → вкладка «Качество» (новая)

5.1. Эталонные задачи



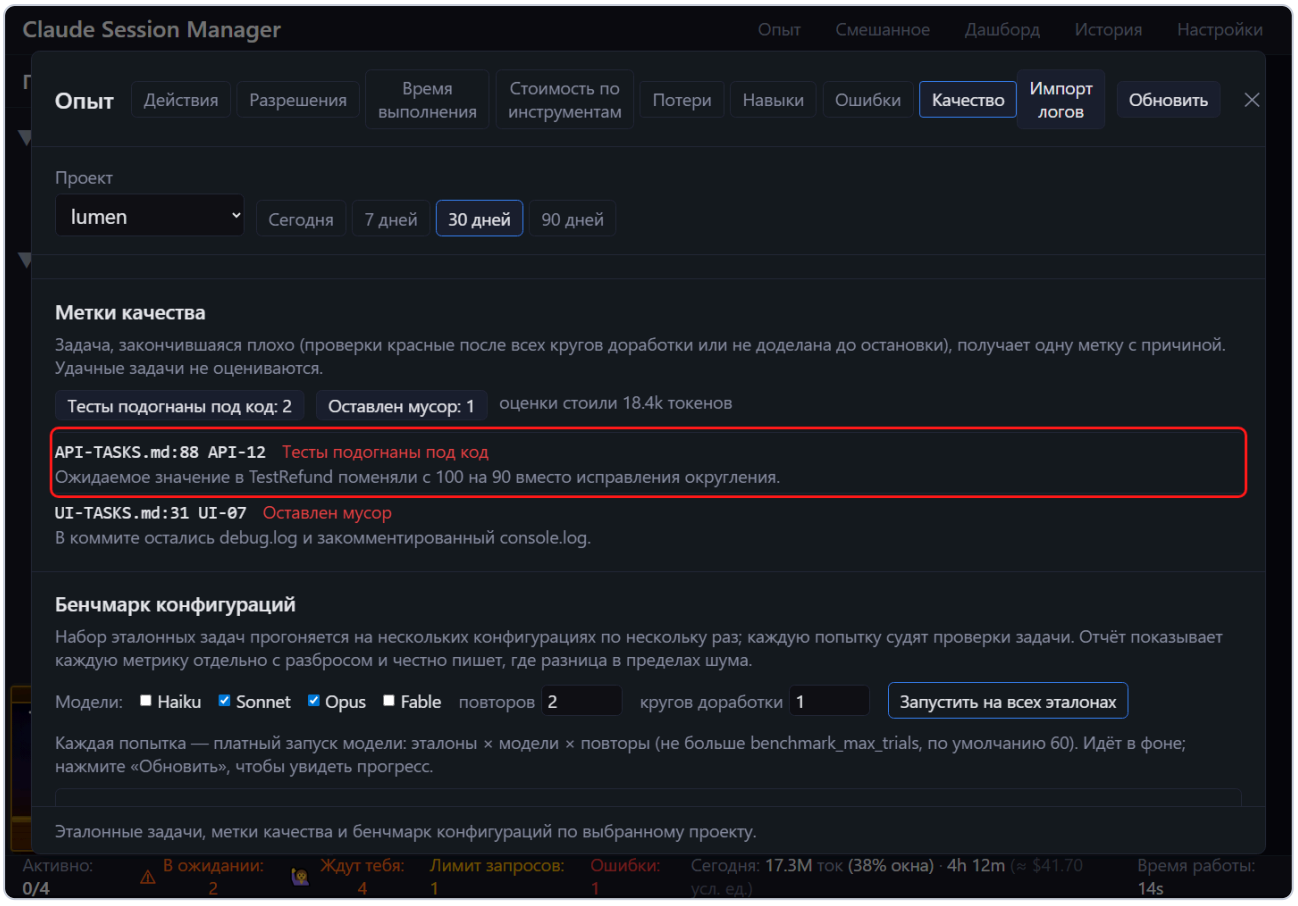
1 вкладка «Качество», 2 выбрать модель и перепрогнать эталон

Когда задача закрылась хорошо, менеджер сохраняет её как **эталон**: текст задачи, коммит, с которого начали, проверки, сколько ушло токенов, времени и кругов, какая была модель. Потом эту же задачу можно **перепрогнать другой моделью** и сравнить с оригиналом в таблице: зелёные/красные проверки, круги, токены, время.

- Перепрогон идёт в **отдельной временной копии** репозитория с того же коммита — твои ветки не трогаются.
- В эталоны попадают задачи, сделанные за **один запуск** (если задача тянулась через несколько запусков, её цифры неполные).
- Если исходный коммит пропал (например, ветку удалили), эталон помечается «недействителен» с причиной — это не ошибка.

> **Как включить**
> Настройки → Общие → **«Эталонные задачи»** + задай **проверки проекта** (Настройки → Проекты). Эталоны начнут копиться с новых задач. Перепрогон — это обычный платный запуск модели.

5.2. Метки качества



Счётчики меток и последние неудачные задачи с причиной

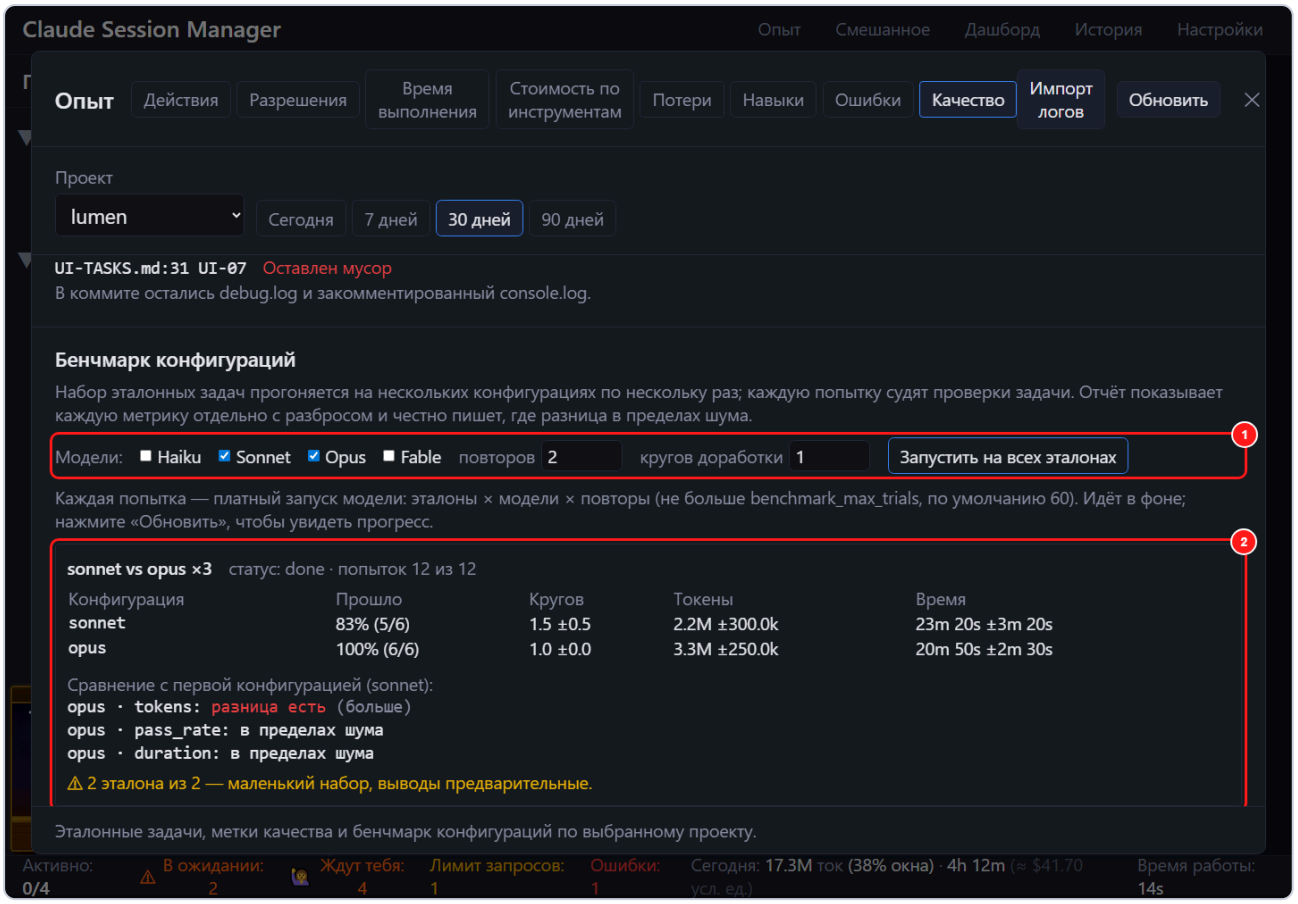
Если задача закончилась **плохо окончательно** (проверки красные после последнего круга доработки или задачу так и не доделали до остановки), самая дешёвая модель (haiku) ставит ей **одну метку** с причиной:

- **Тесты подогнаны под код** — тест «починили», поменяв ожидание, а не код;
- **Нарушен протокол работы** — не по правилам проекта;
- **Оставлен мусор** — лишние файлы, отладочный вывод, недоделки;
- **Сделана не та задача**.

Удачные задачи не оцениваются вообще. Токены «судьи» приписываются к оцениваемому запуску — видно, сколько стоили оценки.

> **Как включить**
> Настройки → Общие → **«Метки качества для неудачных задач»**. Модель-судью можно сменить в конфиге: `quality_label_model` (пусто = haiku).

5.3. Бенчмарк конфигураций



1 форма запуска: модели, повторы, круги доработки, 2 отчёт последнего прогона

Хочешь понять, правда ли opus лучше sonnet **на твоих задачах**? Бенчмарк прогоняет набор эталонных задач на нескольких конфигурациях, каждую по нескольку раз, и каждую попытку судят проверки задачи.

1. Отметь модели, задай число **повторов** и **кругов доработки** 1.
2. Нажми **«Запустить на всех эталонах»**. Прогон идёт в фоне во временных копиях репозитория; жми «Обновить», чтобы видеть прогресс.
3. В отчёте 2 у каждой конфигурации своя строка: сколько прошло, круги, токены, время — со средним и разбросом. Ниже сравнение с первой конфигурацией: **«разница есть»**, **«в пределах шума»** или **«не различить»** (при одном повторе всегда «не различить»). Единой «оценки» нарочно нет.

> **Внимание: это стоит денег**
> Каждая попытка — платный запуск модели: эталоны × модели × повторы. Верхний предел одного прогона — `benchmark_max_trials` (по умолчанию 60). Одновременно идёт только один бенчмарк. Через MCP доступны также тонкие настройки (effort, без скиллов, без запретов, без доработки) — `run_benchmark`.

6. Память о зацикливаниях

Детектор циклов и раньше ловил агента, который бегает по кругу, но помнил только последние ~20 шагов и только в пределах одного запуска. С этой галочкой он:

- ловит повтор, даже если он «размазан» по запуску дальше окна (по умолчанию — один и тот же вызов 6 раз; меняется `loop_far_threshold`);
- запоминает каждый цикл и то, **что помогло из него выйти**;
- если такой же цикл повторится — в подсказке агенту (или в заметке при перезапуске) будет «в прошлый раз помогло вот это».

> **Как включить**
> Настройки → Общие → **«Память о зацикливаниях»**. Отдельного экрана нет — подсказки видны в логе сессии. Токенов не тратит.

7. Отчёт о потерях и тариф — ещё раз коротко

Оба работают сразу: потери — на вкладке «Опыт → Потери», тариф — в статус-баре, на карточке сессии и в Дашборде. Если пользуешься подпиской, просто впиши размер окна лимита в «Тарифе» — и увидишь, какую долю окна съедает день и сколько из этого ушло впустую.

8. Строгие проверки (круги доработки)

Бывает, что сессия говорит «задача готова», а сборка красная. Строгие проверки ловят это сразу:

1. Сессия закрыла задачу.
2. Менеджер запускает **проверки проекта** в рабочей папке сессии.
3. Если красно — текст ошибки уходит этой же сессии следующим сообщением: «почини». Так до N раз.
4. Если и после N кругов красно — сессия останавливается с понятной ошибкой и появляется в «Ждут тебя» как «Ворота красные после всех кругов правок».

> **Как включить**
> Настройки → Сессии → выбери сессию → **«Кругов доработки по красным проверкам»** (например, 2). Нужны **проверки проекта** на вкладке «Проекты» — без них поле подскажет, что ничего не будет. Работает для сессий Claude.

> **На что влияет**
> Каждый круг доработки — это ещё один ход модели (токены). Зато задача реально заканчивается зелёной, а не «на словах».

9. Ночной прогон проверок

Можно попросить менеджер каждую ночь проверять проект:

- в заданное время (например, `03:00`) он запускает проверки в папке проекта;
- если всё **зелёное** — ничего не делает;

- если **красное** — создаёт задачу с текстом ошибки в `SUGGESTED-TASKS.md` и кладёт указатель в очередь. Утром разработчик возьмёт её сам;
- одна и та же поломка — одна задача, пока её указатель в очереди; сессии сам он **не запускает**;
- если менеджер был выключен в это время — прогон догонит при следующем запуске и не повторится дважды за ночь.

> **Как включить**
> Настройки → Проекты → **«Ночной прогон проверок (ЧЧ:ММ)»** и при желании **«Очередь для задачи о красном прогоне»** (по умолчанию — очередь первой сессии).

> **Обрати внимание**
> Время хранится только на этой машине (`config.local.toml`), чтобы коллега, склонировавший проект, не получил чужой ночной запуск команд. Менеджер должен быть запущен.

10. Мелкие улучшения, которые работают сами

- **Лента сессии:** shell-команды с heredoc (`cat > x.py <<EOF`) больше не превращаются в кашу из строк скрипта — видно, какие команды реально запускались, а программы названы без длинного пути.
- **Подбор модели при запуске:** пока Claude анализирует задачу, видно спиннер, счётчик секунд и подсказку. Если анализ идёт дольше 30 секунд — подсказка, что можно не ждать, и заметная кнопка «выбрать вручную».
- **README** теперь объясняет, как устроены сессии, очереди и как настроить проект с нуля.
- Для разработчиков: починен мост Playwright-тестов (CORS для заголовка уровня доступа).

11. Полезно знать

Вопрос	Ответ
Включил галочку — где результат?	Почти всё — в окне «Опыт» на вкладках «Потери» и «Качество» **при выбранном проекте**. «Ждут тебя» — в статус-баре.
Нужно перезапускать менеджер после галочки?	Нет. Только один раз — после обновления версии (шаг 0).
Можно править конфиг руками?	Можно, но только при закрытом менеджере: окно настроек при сохранении перезапишет ручные правки. Все ключи описаны в `config.example.toml` (секция `[optimization]`).
Что попадёт в мой репозиторий?	Только то, что ты подтвердил: хуки в `.claude/`, файлы в `.claude-manager/learned/`, задачи в `SUGGESTED-TASKS.md` и указатели в очереди. Ночной прогон пишет задачу сам — но только при красных проверках.
Что тратит токены?	Перепрогон эталонов, бенчмарк, круги доработки строгих проверок, немного — метки качества. Остальное бесплатно.
Всё это доступно агентам?	Да, через MCP-инструменты `cm`: `get_waste_report`, `get_reference_report`, `replay_reference_case`, `get_task_labels`, `run_benchmark`, `get_benchmark_report` и др.